

The Ultimate Web Development Handbook



Arrays in JavaScript

An **array** is a collection of items stored in a single variable. It lets you store multiple values — like a list of names, numbers, or even other arrays.

1. Creating Arrays

```
let fruits = ["apple", "banana", "mango"];  
let numbers = [10, 20, 30, 40];
```

You can mix data types, but it's better to keep arrays consistent.

```
let mixed = [1, "hello", true];
```

2. Accessing and Modifying Elements

Arrays use **zero-based indexing**.

```
let fruits = ["apple", "banana", "mango"];  
console.log(fruits[0]); // "apple"  
fruits[1] = "orange";    // change "banana" to "orange"  
console.log(fruits);     // ["apple", "orange", "mango"]
```

3. Array Length

```
let colors = ["red", "green", "blue"];  
console.log(colors.length); // 3
```

4. Common Array Methods

a. **push()** – Add item at the end

```
colors.push("yellow");
```

b. **pop()** – Remove last item

```
colors.pop();
```

c. **shift()** – Remove first item

```
colors.shift();
```

d. **unshift()** – Add item at the beginning

```
colors.unshift("pink");
```

5. Looping Through Arrays

Using `for` loop:

```
let fruits = ["apple", "banana", "mango"];
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

Using `for...of` loop:

```
for (let fruit of fruits) {
  console.log(fruit);
}
```

6. `map()` – Transform Array

Creates a new array by applying a function to each item.

```
let numbers = [1, 2, 3];
let doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6]
```

7. `filter()` – Filter Items

Returns a new array of items that match a condition.

```
let scores = [30, 50, 90, 20];
let passed = scores.filter(score => score >= 50);
console.log(passed); // [50, 90]
```

8. Other Useful Methods

Method	Description
<code>includes()</code>	Checks if an item exists in the array
<code>indexOf()</code>	Finds index of an item
<code>slice()</code>	Extracts part of the array
<code>join()</code>	Joins array into a string

Example:

```
let names = ["Ali", "Sara", "John"];
console.log(names.includes("Sara")); // true
console.log(names.indexOf("John")); // 2
console.log(names.join(" - "));      // Ali - Sara - John
```

Summary

- Arrays store lists of data in one variable.
- Use indexing to access or update items.
- Common methods like `push`, `pop`, `map`, `filter` make array handling easier.
- Looping is essential for displaying or processing lists.

Common Array Methods

a. `push()` – Add item at the end

```
colors.push("yellow");
```

b. `pop()` – Remove last item

```
colors.pop();
```

c. `shift()` – Remove first item

```
colors.shift();
```

d. `unshift()` – Add item at the beginning

```
colors.unshift("pink");
```

Looping Through Arrays

Using `for` loop:

```
let fruits = ["apple", "banana", "mango"];
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

Using `for...of` loop:

```
for (let fruit of fruits) {
  console.log(fruit);
}
```

`map()` – Transform Array

Creates a new array by applying a function to each item.

```
let numbers = [1, 2, 3];
let doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6]
```

`filter()` – Filter Items

Returns a new array of items that match a condition.

```
let scores = [30, 50, 90, 20];
let passed = scores.filter(score => score >= 50);
console.log(passed); // [50, 90]
```

Other Useful Methods

Method	Description
<code>includes()</code>	Checks if an item exists in the array
<code>indexOf()</code>	Finds index of an item
<code>slice()</code>	Extracts part of the array
<code>join()</code>	Joins array into a string

Example:

```
let names = ["Ali", "Sara", "John"];
console.log(names.includes("Sara")); // true
console.log(names.indexOf("John")); // 2
console.log(names.join(" - "));      // Ali - Sara - John
```

Summary

- Arrays store lists of data in one variable.
- Use indexing to access or update items.
- Common methods like `push`, `pop`, `map`, `filter` make array handling easier.
- Looping is essential for displaying or processing lists.

Strings in JavaScript

What is a String?

A **string** is a sequence of characters used to represent text. It can contain letters, numbers, symbols, or even be empty.

In JavaScript, strings are written inside **single quotes**, **double quotes**, or **backticks**.

```
let single = 'Hello';  
let double = "World";  
let template = `Hello World`;
```

All three are valid, but backticks (```) are useful for string interpolation (covered later).

Declaring Strings

```
let message = "Welcome to JavaScript!";  
let name = 'Salman';
```

String Length

You can check how many characters are in a string using the `.length` property.

```
let msg = "Hello";  
console.log(msg.length); // 5
```

Multiline Strings

Using `\n` (newline character):

```
let text = "Line 1\nLine 2\nLine 3";  
console.log(text);
```

Using template literals (backticks):

```
let multiline = `This is line 1  
This is line 2  
This is line 3`;  
console.log(multiline);
```

String Indexing

Strings are indexed like arrays — the first character is at position `0`.

```
let word = "JavaScript";  
console.log(word[0]); // "J"  
console.log(word[4]); // "S"
```

Strings are Immutable

You cannot change a specific character in a string directly.

```
let greeting = "Hello";  
greeting[0] = "Y"; // This will not work  
console.log(greeting); // Still "Hello"
```

To change a string, you have to **create a new one**.

Concatenation (Combining Strings)

Using `+` operator:

```
let firstName = "Salman";  
let lastName = "Khan";  
let fullName = firstName + " " + lastName;  
console.log(fullName); // "Salman Khan"
```

Summary

- Strings are sequences of characters.
- Use single, double, or backticks to define them.
- Strings are immutable.
- You can access characters using indices.
- Use `+` for concatenation or backticks for string interpolation.

Template Literals in JavaScript

Template literals are a way to work with strings in JavaScript that allow for easier multi-line strings and string interpolation.

```
let name = "Salman";  
let message = `Hello, ${name}!`;   
console.log(message); // "Hello, Salman!"
```

Template literals are enclosed in backticks (```) and can contain placeholders for variables or expressions, which are wrapped in `${}` .

Multiline Strings with Template Literals

You can create multi-line strings without using escape characters:

```
let multiline = `This is line 1  
This is line 2  
This is line 3`;  
console.log(multiline);
```

Summary

- Template literals use backticks (```).
- They allow for string interpolation with `${}` .
- Multi-line strings can be created easily without escape characters.

String Methods in JavaScript

JavaScript provides many built-in methods to **work with strings** — to inspect, modify, or extract information from them.

All string methods return a **new string** or value. The original string remains unchanged.

1. `length`

Returns the number of characters in the string.

```
let text = "JavaScript";  
console.log(text.length); // 10
```

2. `toUpperCase()` and `toLowerCase()`

Convert a string to upper or lower case.

```
let name = "Ali";  
console.log(name.toUpperCase()); // "ALI"  
console.log(name.toLowerCase()); // "ali"
```

3. `trim()`

Removes extra whitespace from both ends of a string.

```
let input = "  Hello World  ";  
console.log(input.trim()); // "Hello World"
```

4. includes()

Checks if a string contains another string.

```
let msg = "Learn JavaScript";  
console.log(msg.includes("Java")); // true  
console.log(msg.includes("Python")); // false
```

5. indexOf() and lastIndexOf()

Returns the index of the first/last occurrence of a substring. Returns `-1` if not found.

```
let str = "banana";  
console.log(str.indexOf("a")); // 1  
console.log(str.lastIndexOf("a")); // 5
```

6. startsWith() and endsWith()

Check if a string starts or ends with a specific value.

```
let title = "Frontend Developer";  
console.log(title.startsWith("Front")); // true  
console.log(title.endsWith("Dev")); // false
```

7. `slice(start, end)`

Extracts part of a string. `end` is not included.

```
let word = "JavaScript";
console.log(word.slice(0, 4)); // "Java"
console.log(word.slice(4));    // "Script"
```

8. `substring(start, end)`

Similar to `slice()`, but cannot accept negative indexes.

```
let text = "Coding";
console.log(text.substring(0, 3)); // "Cod"
```

9. `replace(old, new)`

Replaces the first occurrence of a substring.

```
let msg = "I love Python";
console.log(msg.replace("Python", "JavaScript")); // "I love JavaScript"
```

Note: Only the first match is replaced. To replace all, use a regular expression with `/g`.

```
let msg = "apple apple apple";
console.log(msg.replace(/apple/g, "banana")); // "banana banana banana"
```

10. split(separator)

Splits a string into an array based on the given separator.

```
let data = "HTML,CSS,JavaScript";  
let parts = data.split(",");  
console.log(parts); // ["HTML", "CSS", "JavaScript"]
```

11. charAt(index)

Returns the character at a specific position.

```
let lang = "JavaScript";  
console.log(lang.charAt(0)); // "J"
```

12. repeat(count)

Repeats a string multiple times.

```
let wow = "ha";  
console.log(wow.repeat(3)); // "hahaha"
```

Summary

Method	Description
<code>.length</code>	String length
<code>.toUpperCase()</code>	Convert to uppercase

Method	Description
<code>.toLowerCase()</code>	Convert to lowercase
<code>.trim()</code>	Remove spaces from both ends
<code>.includes()</code>	Check if string contains a value
<code>.indexOf()</code>	Index of first occurrence
<code>.lastIndexOf()</code>	Index of last occurrence
<code>.startsWith()</code>	Check if string starts with a substring
<code>.endsWith()</code>	Check if string ends with a substring
<code>.slice()</code>	Extract part of a string
<code>.replace()</code>	Replace a part of the string
<code>.split()</code>	Convert string to array
<code>.charAt()</code>	Get character at specific position
<code>.repeat()</code>	Repeat a string multiple times